# Actividad 2 Optimización Matemática

Juan Camilo Tobar Valencia

Roger Rafael Lopesierra Olivera

Santiago García Restrepo

Jesús Andrés Jiménez Pachón

Docente

Whady Felipe Flórez Escobar

Universidad Pontificia Bolivariana

Maestría en Ciencias Naturales y Matemáticas

2026

# Ejercicio 1 — Bisección para minimizar $f(x) = x^4 - 3x^2 + x$

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Use bisección para minimizar $f(x) = x^4 - 3x^2 + x$ en un intervalo donde $f'$ cambie de signo. Verifique con `scipy.optimize.brentq`.

## 1. Definiciones necesarias

### 1.1. Idea del método de bisección para minimizar

Un mínimo de una función $f$ diferenciable ocurre en un punto donde $f'(x^*) = 0$, es decir, en una **raíz de la derivada**. Si se dispone de un intervalo $[a, b]$ tal que $f'(a)$ y $f'(b)$ tienen signos opuestos, el Teorema del Valor Intermedio garantiza que existe al menos un punto en $(a, b)$ donde $f'$ se anula, y por tanto un mínimo (o máximo) dentro del intervalo.

### 1.2. Algoritmo

1. Calcular el punto medio $c = \dfrac{a+b}{2}$ y evaluar $f'(c)$.
2. Si $f'(c)$ tiene el mismo signo que $f'(a)$, actualizar $a \leftarrow c$; en caso contrario, $b \leftarrow c$.
3. Repetir hasta que el ancho del intervalo $b - a$ sea suficientemente pequeño.

### 1.3. Convergencia

La convergencia es **lineal**: el intervalo se reduce exactamente a la mitad en cada iteración, sin importar qué tan complicada sea $f$, siempre que se cumpla la condición de signos opuestos.

## 2. Planteamiento del problema

$$f(x) = x^4 - 3x^2 + x, \qquad f'(x) = 4x^3 - 6x + 1. \tag{1}$$

### 2.1. Localización de los puntos críticos (referencia teórica)

Resolviendo $f'(x) = 0$ se obtienen tres raíces reales, clasificadas con $f''(x) = 12x^2 - 6$:
Se busca $x_1$, el mínimo global, mediante bisección.

| Punto crítico | $f''(x)$ | Clasificación | $f(x)$ |
|---|---|---|---|
| $x_1 \approx -1{,}300840$ | $14{,}31 > 0$ | Mínimo local (global) | $-3{,}513905$ |
| $x_2 \approx 0{,}169938$ | $-5{,}65 < 0$ | Máximo local | $0{,}084135$ |
| $x_3 \approx 1{,}130901$ | $9{,}35 > 0$ | Mínimo local | $-1{,}070230$ |

Cuadro 1: Puntos críticos de $f(x) = x^4 - 3x^2 + x$.

## 3. Elección del intervalo inicial

Se elige $[a, b] = [-2, 0]$.

$$f'(-2) = 4(-8) - 6(-2) + 1 = -32 + 12 + 1 = -19, \tag{2}$$

$$f'(0) = 1. \tag{3}$$

$$\boxed{f'(-2) = -19 < 0 \qquad \text{y} \qquad f'(0) = 1 > 0}$$

Hay cambio de signo, por lo que existe un mínimo dentro de $(-2, 0)$.

## 4. Tabla completa de las 22 iteraciones

Se aplica el algoritmo con tolerancia $\text{tol} = 10^{-6}$, deteniendo cuando $b - a < \text{tol}$ o $|f'(c)| < \text{tol}$.

| it | $a$ | $b$ | $c$ | $f'(c)$ | ancho $(b-a)$ |
|---|---|---|---|---|---|
| 1 | $-2{,}0000000$ | $0{,}0000000$ | $-1{,}0000000$ | $3{,}0000000$ | $2{,}0000 \times 10^{0}$ |
| 2 | $-2{,}0000000$ | $-1{,}0000000$ | $-1{,}5000000$ | $-3{,}5000000$ | $1{,}0000 \times 10^{0}$ |
| 3 | $-1{,}5000000$ | $-1{,}0000000$ | $-1{,}2500000$ | $0{,}6875000$ | $5{,}0000 \times 10^{-1}$ |
| 4 | $-1{,}5000000$ | $-1{,}2500000$ | $-1{,}3750000$ | $-1{,}1484375$ | $2{,}5000 \times 10^{-1}$ |
| 5 | $-1{,}3750000$ | $-1{,}2500000$ | $-1{,}3125000$ | $-0{,}1689453$ | $1{,}2500 \times 10^{-1}$ |
| 6 | $-1{,}3125000$ | $-1{,}2500000$ | $-1{,}2812500$ | $0{,}2742920$ | $6{,}2500 \times 10^{-2}$ |
| 7 | $-1{,}3125000$ | $-1{,}2812500$ | $-1{,}2968750$ | $0{,}0564728$ | $3{,}1250 \times 10^{-2}$ |
| 8 | $-1{,}3125000$ | $-1{,}2968750$ | $-1{,}3046875$ | $-0{,}0552807$ | $1{,}5625 \times 10^{-2}$ |
| 9 | $-1{,}3046875$ | $-1{,}2968750$ | $-1{,}3007812$ | $0{,}0008342$ | $7{,}8125 \times 10^{-3}$ |
| 10 | $-1{,}3046875$ | $-1{,}3007812$ | $-1{,}3027344$ | $-0{,}0271636$ | $3{,}9063 \times 10^{-3}$ |
| 11 | $-1{,}3027344$ | $-1{,}3007812$ | $-1{,}3017578$ | $-0{,}0131498$ | $1{,}9531 \times 10^{-3}$ |
| 12 | $-1{,}3017578$ | $-1{,}3007812$ | $-1{,}3012695$ | $-0{,}0061541$ | $9{,}7656 \times 10^{-4}$ |
| 13 | $-1{,}3012695$ | $-1{,}3007812$ | $-1{,}3010254$ | $-0{,}0026590$ | $4{,}8828 \times 10^{-4}$ |
| 14 | $-1{,}3010254$ | $-1{,}3007812$ | $-1{,}3009033$ | $-0{,}0009121$ | $2{,}4414 \times 10^{-4}$ |
| 15 | $-1{,}3009033$ | $-1{,}3007812$ | $-1{,}3008423$ | $-0{,}0000389$ | $1{,}2207 \times 10^{-4}$ |
| 16 | $-1{,}3008423$ | $-1{,}3007812$ | $-1{,}3008118$ | $0{,}0003977$ | $6{,}1035 \times 10^{-5}$ |
| 17 | $-1{,}3008423$ | $-1{,}3008118$ | $-1{,}3008270$ | $0{,}0001794$ | $3{,}0518 \times 10^{-5}$ |
| 18 | $-1{,}3008423$ | $-1{,}3008270$ | $-1{,}3008347$ | $0{,}0000702$ | $1{,}5259 \times 10^{-5}$ |
| 19 | $-1{,}3008423$ | $-1{,}3008347$ | $-1{,}3008385$ | $0{,}0000157$ | $7{,}6294 \times 10^{-6}$ |
| 20 | $-1{,}3008423$ | $-1{,}3008385$ | $-1{,}3008404$ | $-0{,}0000116$ | $3{,}8147 \times 10^{-6}$ |
| 21 | $-1{,}3008404$ | $-1{,}3008385$ | $-1{,}3008394$ | $0{,}0000020$ | $1{,}9073 \times 10^{-6}$ |
| 22 | $-1{,}3008404$ | $-1{,}3008394$ | $-1{,}3008399$ | $-0{,}0000048$ | $9{,}5367 \times 10^{-7}$ |

Cuadro 2: Iteraciones completas de bisección sobre $f'(x)$ en $[-2, 0]$.

En la iteración **22**, el ancho $9{,}54 \times 10^{-7} < 10^{-6}$, por lo que el algoritmo se detiene.

$$\boxed{x^* \approx -1{,}3008399}$$

### 4.1. Verificación del patrón de reducción del ancho

El ancho inicial es $b - a = 2{,}0$, y en cada iteración se divide entre 2. Tras 22 iteraciones:

$$\frac{2{,}0}{2^{22}} = \frac{2{,}0}{4194304} \approx 4{,}77 \times 10^{-7},$$

valor consistente con el ancho final observado en la tabla.

## 5. Cálculo del valor óptimo

$$f(x^*) = (-1{,}3008399)^4 - 3(-1{,}3008399)^2 + (-1{,}3008399). \tag{4}$$

$$\boxed{f(x^*) \approx -3{,}513905.}$$

## 6. Verificación con `scipy.optimize.brentq` (con diagnóstico completo)

Se resuelve el mismo problema (raíz de $f'$ en $[-2, 0]$) usando `full_output=True`, que además de la raíz entrega información detallada de la convergencia:

```python
from scipy.optimize import brentq

fprime = lambda x: 4*x**3 - 6*x + 1

# Pasamos full_output=True
x_star, info = brentq(fprime, -2, 0, xtol=1e-6, full_output=True)

print(f"Raiz hallada: {x_star}")
print(f"Numero de iteraciones: {info.iterations}")
print(f"Evaluaciones de la funcion: {info.function_calls}")
print(f"Convergio?: {info.converged}")
```

### Resultado obtenido

```
Raiz hallada: -1.3008395659988075
Numero de iteraciones: 10
Evaluaciones de la funcion: 11
Convergio?: True
```

con $f(x^*) = -3{,}513905038934789$.

### 6.1. Comparación completa entre los enfoques

| Ambos métodos coinciden en $x^*$ hasta la tolerancia especificada. ✓ |
|---|

| Método | $x^*$ | $f(x^*)$ | Iteraciones | Evaluaciones de $f'$ |
|---|---|---|---|---|
| Bisección manual | $-1{,}3008399$ | $-3{,}513905$ | 22 | 22 |
| `brentq` (scipy) | $-1{,}3008396$ | $-3{,}513905$ | **10** | **11** |

Cuadro 3: Comparación de bisección manual vs. `brentq`.

### 6.2. Análisis de la eficiencia

El resultado numérico confirma la ventaja teórica de Brent frente a bisección pura: `brentq` necesita solo 10 iteraciones (11 evaluaciones de $f'$) frente a las 22 iteraciones (22 evaluaciones) de la bisección manual, es decir, aproximadamente **la mitad de evaluaciones** para alcanzar la misma tolerancia $10^{-6}$.

Esto ocurre porque `brentq` combina tres estrategias:

- **Bisección pura** (como respaldo garantizado, igual a la implementada a mano).
- **Método de la secante** (usa una interpolación lineal entre dos puntos para converger más rápido cuando la función se comporta bien).
- **Interpolación cuadrática inversa** (ajusta una parábola a tres puntos para acelerar aún más la convergencia cerca de la raíz).

El campo `info.converged=True` confirma que el algoritmo cumplió el criterio de parada de forma segura, y `info.function_calls=11` (una evaluación más que las iteraciones) refleja la evaluación inicial en los extremos del intervalo antes de comenzar el refinamiento.

## 7. Verificación con el criterio de segunda derivada

$$f''(x^*) = 12(-1{,}3008399)^2 - 6 \approx 14{,}31 > 0. \tag{5}$$

Se confirma que $x^* \approx -1{,}300840$ es un **mínimo local**, y comparando con el otro mínimo local ($f(x_3) \approx -1{,}070230$), también es el **mínimo global** de $f$, pues $-3{,}513905 < -1{,}070230$.

## 8. Resultado final

$$\boxed{x^* \approx -1{,}300840, \qquad f(x^*) \approx -3{,}513905}$$

| Aspecto | Bisección manual | `brentq` con diagnóstico |
|---|---|---|
| $x^*$ | $-1{,}3008399$ | $-1{,}3008396$ |
| $f(x^*)$ | $-3{,}513905$ | $-3{,}513905$ |
| Iteraciones | 22 | 10 |
| Evaluaciones de $f'$ | 22 | 11 |
| Convergencia garantizada | Sí (lineal) | Sí (superlineal) |

Cuadro 4: Resumen final comparativo.

**Interpretación:** el diagnóstico de `full_output=True` demuestra cuantitativamente por qué en la práctica se prefiere `brentq` sobre bisección pura: consigue la misma precisión con menos de la mitad de las evaluaciones de la derivada, gracias a que aprovecha información adicional (secante e interpolación cuadrática) en lugar de descartar siempre la mitad del intervalo de forma ciega.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*

# Ejercicio 2 — Comparación de Bisección y Búsqueda Áurea

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Compare, para tres tolerancias distintas ($10^{-3}$, $10^{-6}$, $10^{-9}$), cuántas iteraciones necesitan bisección y búsqueda áurea. ¿La diferencia crece o se mantiene constante?

## 1. Definiciones necesarias

### 1.1. Tasa de reducción de cada método

**Bisección:** en cada iteración el intervalo se reduce exactamente a la mitad:

$$b_n - a_n = \frac{b_0 - a_0}{2^n}. \tag{1}$$

**Búsqueda áurea:** en cada iteración el intervalo se reduce por el factor de la razón áurea $\varphi = \dfrac{\sqrt{5}-1}{2} \approx 0{,}618$:

$$b_n - a_n = (b_0 - a_0) \cdot \varphi^n. \tag{2}$$

Como $\varphi \approx 0{,}618 > 0{,}5$, la búsqueda áurea reduce el intervalo **más lentamente** que la bisección en cada paso, aunque no necesita la derivada.

### 1.2. Fórmula del número de iteraciones necesarias

Para alcanzar una tolerancia tol a partir de un ancho inicial $w_0 = b_0 - a_0$, se despeja $n$ de la condición $w_0 \cdot r^n \leq \text{tol}$:

$$n \geq \frac{\ln(w_0/\text{tol})}{\ln(1/r)}, \tag{3}$$

donde $r = 0{,}5$ para bisección y $r = \varphi \approx 0{,}618$ para búsqueda áurea.

## 2. Planteamiento del problema

Se usa la función de referencia de la Clase 2:

$$f(x) = x^2 - 2x + 3, \qquad f'(x) = 2x - 2, \tag{4}$$

con mínimo exacto en $x^* = 1$, sobre el intervalo inicial $[a, b] = [-2, 5]$, con ancho $w_0 = 7$.

## 3. Cálculo teórico del número de iteraciones esperado

**Para bisección ($r = 0{,}5$)**

$$n_{\text{bisec}} \geq \frac{\ln(7/\text{tol})}{\ln 2}. \tag{5}$$

**Para búsqueda áurea** ($r = \varphi \approx 0{,}618$)

$$n_{\text{áurea}} \geq \frac{\ln(7/\text{tol})}{\ln(1/0{,}618)} = \frac{\ln(7/\text{tol})}{\ln(1{,}618)}. \tag{6}$$

| tol | $n_{\text{bisec}}$ teórico | $n_{\text{áurea}}$ teórico |
|---|---|---|
| $10^{-3}$ | $12{,}77 \to \lceil\cdot\rceil = 13$ | $18{,}40 \to \lceil\cdot\rceil = 19$ |
| $10^{-6}$ | $22{,}74 \to \lceil\cdot\rceil = 23$ | $32{,}75 \to \lceil\cdot\rceil = 33$ |
| $10^{-9}$ | $32{,}70 \to \lceil\cdot\rceil = 33$ | $47{,}11 \to \lceil\cdot\rceil = 48$ |

Cuadro 1: Predicciones teóricas del número de iteraciones.

## 4. Implementación y conteo real de iteraciones

```python
import numpy as np

f = lambda x: x**2 - 2*x + 3
fprime = lambda x: 2*x - 2

def biseccion_count(fprime, a, b, tol):
    it = 0
    fa = fprime(a)
    while True:
        it += 1
        c = 0.5*(a+b)
        fc = fprime(c)
        ancho = b - a
        if abs(fc) < tol or ancho < tol:
            return it, c
        if np.sign(fc) == np.sign(fa):
            a = c
            fa = fc
        else:
            b = c

def aurea_count(f, a, b, tol):
    PHI = (5**0.5 - 1)/2
    x1 = b - PHI*(b-a)
    x2 = a + PHI*(b-a)
    f1, f2 = f(x1), f(x2)
    it = 0
    while (b-a) > tol:
        it += 1
        if f1 < f2:
            b = x2
            x2, f2 = x1, f1
            x1 = b - PHI*(b-a)
            f1 = f(x1)
        else:
            a = x1
```

```
            x1, f1 = x2, f2
            x2 = a + PHI*(b-a)
            f2 = f(x2)
    return it, 0.5*(a+b)

# Definicion del intervalo
a, b = -2, 5

# Iteracion e impresion segun el formato de la imagen
tolerancias = [1e-3, 1e-6, 1e-9]

for tol in tolerancias:
    it_bis, x_bis = biseccion_count(fprime, a, b, tol)
    it_aur, x_aur = aurea_count(f, a, b, tol)

    print(f"tol={tol:.0e} | biseccion: it={it_bis}, x*={x_bis:.8f} | aurea
        : it={it_aur}, x*={x_aur:.8f}")
```

**Salida obtenida**

```
tol=1e-03 | biseccion: it=13, x*=1.00012207 | aurea: it=19, x*=1.00022655
tol=1e-06 | biseccion: it=22, x*=1.00000024 | aurea: it=33, x*=0.99999983
tol=1e-09 | biseccion: it=31, x*=1.00000000 | aurea: it=48, x*=1.00000002
```

# 5. Tabla comparativa de resultados

| Tolerancia | Iteraciones bisección | Iteraciones áurea | Diferencia (áurea − bisección) |
|---|---|---|---|
| $10^{-3}$ | 13 | 19 | 6 |
| $10^{-6}$ | 22 | 33 | 11 |
| $10^{-9}$ | 31 | 48 | 17 |

Cuadro 2: Comparación de iteraciones reales entre ambos métodos.

*Nota sobre la pequeña discrepancia con la teoría:* el valor real de bisección para $10^{-9}$ es 31 en lugar del 33 teórico, porque el criterio de parada usa `abs(fc) <tol` o `ancho <tol`, lo que a veces detiene el algoritmo un paso antes al cumplirse la primera condición sobre el valor de $f'(c)$ en vez de sobre el ancho. Esto no afecta la conclusión cualitativa.

# 6. ¿La diferencia crece o se mantiene constante?

## 6.1. Análisis directo de los incrementos

$$\Delta_{10^{-6}-10^{-3}}: \text{bisec} = 22 - 13 = 9, \qquad \text{áurea} = 33 - 19 = 14. \tag{7}$$

$$\Delta_{10^{-9}-10^{-6}}: \text{bisec} = 31 - 22 = 9, \qquad \text{áurea} = 48 - 33 = 15. \tag{8}$$

**Diferencia entre métodos por cada tolerancia**

$$6 \;\to\; 11 \;\to\; 17. \tag{9}$$

La diferencia entre bisección y áurea **crece** a medida que la tolerancia se hace más exigente.

### 6.2. Justificación matemática

Cada vez que la tolerancia se reduce en un factor de 1000 (tres órdenes de magnitud), **ambos** métodos necesitan un número **constante** de iteraciones adicionales, pero ese número constante es **distinto** para cada método:

$$\Delta n_{\text{bisec}} = \frac{\ln(1000)}{\ln 2} \approx \frac{6{,}908}{0{,}693} \approx 9{,}97 \approx 10 \text{ iteraciones extra por cada factor 1000,} \tag{10}$$

$$\Delta n_{\text{áurea}} = \frac{\ln(1000)}{\ln(1{,}618)} \approx \frac{6{,}908}{0{,}481} \approx 14{,}36 \approx 14\text{–}15 \text{ iteraciones extra por cada factor 1000.} \tag{11}$$

Como $\Delta n_{\text{áurea}} > \Delta n_{\text{bisec}}$ en **cada** intervalo de tolerancia, la **brecha acumulada** entre ambos métodos se hace cada vez mayor: 6, 11, 17, ...

Este patrón no es casual: la razón entre los incrementos es constante e igual a

$$\frac{\Delta n_{\text{áurea}}}{\Delta n_{\text{bisec}}} = \frac{\ln 2}{\ln(1/\varphi)} = \frac{0{,}6931}{0{,}4812} \approx 1{,}44, \tag{12}$$

lo que significa que la búsqueda áurea siempre necesita aproximadamente un **44 % más de iteraciones** que la bisección para ganar la misma cantidad de dígitos de precisión, y esa brecha porcentual se traduce en una **brecha absoluta creciente** en términos de iteraciones.

## 7. Verificación gráfica del crecimiento (interpretación numérica)

| tol | $\log_{10}(1/\text{tol})$ | it. bisección | it. áurea | Razón áurea/bisección |
|---|---|---|---|---|
| $10^{-3}$ | 3 | 13 | 19 | 1.46 |
| $10^{-6}$ | 6 | 22 | 33 | 1.50 |
| $10^{-9}$ | 9 | 31 | 48 | 1.55 |

Cuadro 3: Razón de iteraciones entre áurea y bisección.

La razón entre iteraciones se mantiene relativamente estable alrededor de 1,44–1,55 (consistente con el valor teórico $\approx 1{,}44$), pero la **diferencia absoluta** (columna final de la tabla anterior) sigue creciendo linealmente con el número de dígitos de precisión exigidos.

## 8. Resultado final

Iteraciones necesarias: bisección $\approx 3{,}32 \log_{10}(w_0/\text{tol})$, áurea $\approx 4{,}78 \log_{10}(w_0/\text{tol})$

| Conclusión | Justificación |
|---|---|
| La diferencia **crece** con la precisión exigida | Cada método tiene una tasa de convergencia lineal distinta ($r = 0{,}5$ vs. $r \approx 0{,}618$) |
| Bisección siempre gana en número de iteraciones | $\ln(2) > \ln(1/\varphi)$, es decir, la bisección reduce el intervalo más agresivamente por paso |
| La ventaja de bisección requiere $f'$ | Búsqueda áurea es la alternativa cuando no se puede calcular o evaluar $f'$ |

Cuadro 4: Resumen de conclusiones.

**Interpretación:** aunque ambos métodos convergen linealmente (geométricamente) y son igual de "seguros" en el sentido de que garantizan converger si la función es unimodal, **bisección es sistemáticamente más eficiente que búsqueda áurea cuando se dispone de la derivada**, y esa ventaja se vuelve más notoria (en términos absolutos de iteraciones ahorradas) mientras más estricta sea la tolerancia requerida. Esto justifica por qué en la práctica, si $f'$ está disponible y es barata de evaluar, se prefiere bisección (o mejor aún, un método tipo Brent) sobre búsqueda áurea.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*

# Ejercicio 3 — Grid Search en 4 variables vs. 2 variables

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Implemente grid search en 4 variables para $f(x_1, x_2, x_3, x_4) = \sum x_i^2$ con $N = 20$ puntos por eje. Mida el tiempo de ejecución y compárelo con el caso $N = 20$, $d = 2$.

## 1. Definiciones necesarias

### 1.1. Idea de grid search (búsqueda exhaustiva)

Evaluar $f$ en todos los puntos de una malla regular y quedarse con el mejor. No requiere derivadas, no requiere que $f$ sea continua, y no le importa cuántos mínimos locales tenga $f$: siempre encuentra el mínimo de la malla, que se acerca al mínimo real si la malla es suficientemente fina.

### 1.2. La maldición de la dimensionalidad

Con $N$ puntos por eje y $d$ variables, el costo es $N^d$ evaluaciones. Con $N = 50$:

$$d = 1 \to 50, \quad d = 2 \to 2\,500, \quad d = 3 \to 125\,000, \quad d = 4 \to 6\,250\,000, \ldots$$

Por eso grid search es práctico en 1–3 variables, y rápidamente inviable más allá.

## 2. Planteamiento del problema

Se define

$$f(x_1, x_2, x_3, x_4) = x_1^2 + x_2^2 + x_3^2 + x_4^2, \tag{1}$$

con $N = 20$ puntos por eje en el rango $[-5, 5]$, y se compara contra el caso análogo con $d = 2$:

$$f(x_1, x_2) = x_1^2 + x_2^2. \tag{2}$$

### 2.1. Cálculo teórico del número de evaluaciones

$$\text{Evaluaciones}(d = 2) = N^2 = 20^2 = 400. \tag{3}$$

$$\text{Evaluaciones}(d = 4) = N^4 = 20^4 = 160\,000. \tag{4}$$

$$\boxed{\frac{\text{Evaluaciones}(d = 4)}{\text{Evaluaciones}(d = 2)} = \frac{N^4}{N^2} = N^2 = 20^2 = 400.}$$

## 3. Implementación en Python

```python
import itertools, numpy as np, timeit

N = 20
rango = np.linspace(-5, 5, N)

def grid_2d_loop():
    mejor = None
    f_best = np.inf
    for punto in itertools.product(rango, repeat=2):
        val = punto[0]**2 + punto[1]**2
        if val < f_best:
            f_best = val
            mejor = punto
    return mejor, f_best

def grid_4d():
    mejor = None
    f_best = np.inf
    for punto in itertools.product(rango, repeat=4):
        val = punto[0]**2 + punto[1]**2 + punto[2]**2 + punto[3]**2
        if val < f_best:
            f_best = val
            mejor = punto
    return mejor, f_best

t2 = timeit.timeit(grid_2d_loop, number=20)/20
t4 = timeit.timeit(grid_4d, number=20)/20

print("tiempo promedio d=2 (bucle):", t2)
print("tiempo promedio d=4 (bucle):", t4)
print("razon tiempo:", t4/t2)
```

### Resultado obtenido

```
tiempo promedio d=2 (bucle): 0.0002972830000044785
tiempo promedio d=4 (bucle): 0.1575149663000019
razon tiempo: 529.8485493612638
```

## 4. Resultados obtenidos

| Caso | Evaluaciones | Tiempo promedio (`number=20`) |
|---|---|---|
| $d = 2$ | 400 | $2{,}973 \times 10^{-4}$ s |
| $d = 4$ | 160 000 | $1{,}575 \times 10^{-1}$ s |

Cuadro 1: Resultados de tiempo y número de evaluaciones.

$$\boxed{\text{razón de tiempo} = \frac{0{,}1575149663}{0{,}0002972830} \approx 529{,}85}$$

# 5. Comparación de costos: teoría vs. medición real

| Magnitud | Valor teórico | Valor medido |
|---|---|---|
| Razón de evaluaciones ($N^4/N^2$) | 400 | 400 (exacto, por conteo) |
| Razón de tiempo de ejecución | $\approx 400$ (esperado) | 529.85 |

Cuadro 2: Comparación entre predicción teórica y medición real.

## 5.1. Explicación de la discrepancia entre 400 y 529.85

El tiempo no crece exactamente igual que el número de evaluaciones, sino un poco más.

Esto ocurre porque, en el bucle de Python, cada evaluación en $d = 4$ involucra ligeramente **más trabajo por evaluación** que en $d = 2$:

- Se debe sumar **4 términos** ($x_1^2 + x_2^2 + x_3^2 + x_4^2$) en lugar de **2 términos** ($x_1^2 + x_2^2$).
- `itertools.product` con `repeat=4` genera tuplas más largas, con un costo de desempaquetado ligeramente mayor.
- El overhead fijo de Python por iteración del bucle (que no depende de $d$) se diluye proporcionalmente menos cuando hay más operaciones aritméticas por iteración, pero el conteo de sumas sí crece.

Por eso la razón real (529,85) es mayor que la razón teórica basada solo en el conteo de evaluaciones (400): el trabajo por evaluación también creció al duplicarse el número de variables sumadas.

# 6. Extrapolación

Usando el tiempo medido de $d = 4$ (0,1575 s) y la razón de crecimiento observada:

| $d$ | Evaluaciones $N^d$ | Tiempo estimado (extrapolado desde datos reales) |
|---|---|---|
| 2 | 400 | $2{,}97 \times 10^{-4}$ s (medido) |
| 3 | 8 000 | $\approx 6{,}8 \times 10^{-3}$ s |
| 4 | 160 000 | $1{,}575 \times 10^{-1}$ s (medido) |
| 5 | 3 200 000 | $\approx 3{,}6$ s |
| 6 | 64 000 000 | $\approx 83$ s ($\approx 1{,}4$ min) |

Cuadro 3: Extrapolación del costo computacional según la dimensión.

Con solo 2 dimensiones más (de 4 a 6), el tiempo estimado pasa de décimas de segundo a más de un minuto.

Esta progresión exponencial confirma por qué grid search se vuelve **completamente imprácticco** más allá de 3–4 variables.

## 7. Resultado final

$$\text{Evaluaciones}(d=2) = 400, \qquad \text{Evaluaciones}(d=4) = 160\,000, \qquad \text{razón} = 400$$

$$\text{Tiempo}(d=2) \approx 2{,}973 \times 10^{-4}\ \text{s}, \qquad \text{Tiempo}(d=4) \approx 1{,}575 \times 10^{-1}\ \text{s}, \qquad \text{razón} \approx 529{,}85$$

**Interpretación:** el número de evaluaciones creció exactamente en el factor teórico $N^2 = 400$ al pasar de 2 a 4 variables, confirmando la fórmula $N^d$. El tiempo de ejecución creció en un factor algo mayor (529,85) porque cada evaluación individual también se volvió más costosa al duplicarse el número de términos sumados. Este ejercicio ilustra de forma concreta por qué **grid search es viable solo hasta 3 o 4 variables**: duplicar la dimensión (de 2 a 4) ya multiplica el costo por cientos, y agregar solo dos variables más (a 6) lo llevaría a más de un minuto de cómputo con el mismo $N$.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*

# Ejercicio 4 — Random Walk con Mecanismo de Reinicio

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Modifique `scripts/05_random_walk_2d.py` agregando un reinicio: si tras 50 pasos no hay mejora, saltar a un punto aleatorio nuevo. Compare el resultado contra la versión sin reinicios.

## 1. Definiciones necesarias

### 1.1. Random walk greedy (versión original del script)

Da un paso aleatorio a partir del punto actual y **solo lo acepta si mejora** $f$; si no mejora, se descarta y se intenta otro paso aleatorio desde el mismo punto. Es un optimizador simple pero puramente **local**: una vez que cae en la cuenca de atracción de un mínimo local, no tiene forma de escapar de él.

### 1.2. Limitación observada en la Clase 2

El material de la Clase 2 muestra que, ante un paisaje con 6 mínimos locales, tanto el random walk greedy como el método Powell de scipy (otro optimizador local derivative-free) quedan atrapados sin encontrar el mínimo global, porque ningún optimizador puramente local tiene forma de "saber" que hay algo mejor del otro lado de una colina si nunca la cruza.

### 1.3. Mecanismo de reinicio

La idea es introducir un contador de pasos sin mejora. Si ese contador alcanza un umbral de **paciencia** (aquí, 50 pasos), el algoritmo abandona su posición actual y **salta a un punto aleatorio nuevo** dentro del dominio, reiniciando el contador. Esto le da al algoritmo la posibilidad de explorar otras cuencas de atracción sin quedar encerrado para siempre en la primera que encuentra. Esta idea es el primer paso hacia las metaheurísticas de la Clase 3 (basin hopping, simulated annealing).

> *Nota sobre el paisaje usado:* el archivo `_landscape.py` original del script no estaba disponible en los archivos adjuntos, por lo que se reconstruyó un paisaje equivalente de 6 pozos gaussianos, consistente con la descripción de la Clase 2 (mínimo global $f \approx -3{,}29$), documentado explícitamente a continuación.

## 2. Reconstrucción del paisaje de 6 pozos

```
import numpy as np

POZOS = [
    ("pozo1", -6.0,  6.0, -1.8),
    ("pozo2",  5.0,  5.5, -2.2),
    ("pozo3", -7.0, -3.0, -2.6),
```

```
    ("pozo4",  6.5, -6.0, -3.29),   # pozo global, profundidad -3.29
    ("pozo5",  0.0,  7.5, -1.5),
    ("pozo6", -3.5, -7.0, -2.0),
]

def f(x, y):
    total = 0.0
    for _, cx, cy, depth in POZOS:
        r2 = (x - cx)**2 + (y - cy)**2
        total = total + depth * np.exp(-r2 / 8.0)
    return total
```

**Verificación del mínimo global** (por malla fina $400 \times 400$):

```
minimo aproximado: (6.491, -5.990), f = -3.2899
```

Consistente con el valor $f \approx -3{,}29$ reportado en la Clase 2.

## 3. Código modificado: random walk greedy con reinicio

```
def random_walk_greedy(f, x0, step=1.0, n_iter=300, seed=0):
    """Version ORIGINAL, sin reinicio (la del script de la Clase 2)."""
    rng = np.random.default_rng(seed)
    x = np.array(x0, dtype=float)
    fx = f(*x)
    trayectoria = [x.copy()]
    aceptados = 0
    for _ in range(n_iter):
        paso = rng.uniform(-step, step, size=2)
        x_new = x + paso
        f_new = f(*x_new)
        if f_new < fx:
            x, fx = x_new, f_new
            trayectoria.append(x.copy())
            aceptados += 1
    return x, fx, np.array(trayectoria), aceptados / n_iter


def random_walk_greedy_restart(f, x0, step=1.0, n_iter=300, seed=0,
                               paciencia=50, dominio=(-10, 10)):
    """Version MODIFICADA: agrega reinicio tras 'paciencia' pasos sin
       mejora."""
    rng = np.random.default_rng(seed)
    x = np.array(x0, dtype=float)
    fx = f(*x)
    x_best, f_best = x.copy(), fx
    trayectoria = [x.copy()]
    aceptados = 0
    reinicios = 0
    sin_mejora = 0

    for _ in range(n_iter):
        paso = rng.uniform(-step, step, size=2)
        x_new = x + paso
```

```
        f_new = f(*x_new)
        if f_new < fx:
            x, fx = x_new, f_new
            trayectoria.append(x.copy())
            aceptados += 1
            sin_mejora = 0
            if fx < f_best:
                x_best, f_best = x.copy(), fx
        else:
            sin_mejora += 1

        # --- MECANISMO DE REINICIO ---
        if sin_mejora >= paciencia:
            x = rng.uniform(dominio[0], dominio[1], size=2)
            fx = f(*x)
            trayectoria.append(x.copy())
            sin_mejora = 0
            reinicios += 1
            if fx < f_best:
                x_best, f_best = x.copy(), fx

    return x_best, f_best, np.array(trayectoria), aceptados / n_iter,
        reinicios
```

La modificación clave respecto al script original es el bloque marcado como **MECANISMO DE REINICIO**: se agrega un contador `sin_mejora` que se resetea cada vez que hay una mejora, y al superar el umbral (`paciencia=50`) fuerza un salto a un punto uniforme aleatorio en todo el dominio.

## 4. Experimento: punto de arranque atrapado en un mínimo local

Se elige deliberadamente un punto de arranque cerca de un **pozo local no-global** ($f \approx -1{,}8$), lejos del pozo global ($f \approx -3{,}29$), para poder observar si el reinicio permite escapar:

```
x0 = np.array([-6.0, 4.0])  # cerca del pozo local, lejos del pozo global

xg, fg, tray_g, tasa_g = random_walk_greedy(f, x0, step=1.5, n_iter=500,
    seed=0)
print(f"[SIN reinicio] x*={xg.round(3)} f*={fg:.4f} "
      f"({len(tray_g)-1} pasos aceptados de 500, tasa={tasa_g*100:.1f}%)")

xr, fr, tray_r, tasa_r, reinicios = random_walk_greedy_restart(
    f, x0, step=1.5, n_iter=500, seed=0, paciencia=50)
print(f"[CON reinicio] x*={xr.round(3)} f*={fr:.4f} "
      f"(tasa aceptacion={tasa_r*100:.1f}%, reinicios={reinicios})")
```

### Resultado obtenido

```
[SIN reinicio] x*=[-5.951  5.954] f*=-1.8124 (9 pasos aceptados de 500,
    tasa=1.8%)
[CON reinicio] x*=[ 6.43  -5.968] f*=-3.2876 (tasa aceptacion=8.8%,
    reinicios=5)
```

## 5. Tabla comparativa del experimento principal

| Versión | $x^*$ | $f(x^*)$ | Pasos aceptados | Reinicios | ¿Global? |
|---|---|---|---|---|---|
| Sin reinicio | $(-5{,}951, 5{,}954)$ | $-1{,}8124$ | 9 de 500 | 0 | No |
| Con reinicio | $(6{,}430, -5{,}968)$ | $-3{,}2876$ | — | 5 | Sí |

Cuadro 1: Comparación de resultados entre ambas versiones.

La versión con reinicio mejora el valor óptimo de $-1{,}81$ a $-3{,}29$, una mejora del $81{,}4\,\%$.

## 6. Verificación de robustez con múltiples semillas

Para confirmar que el resultado no es casualidad de una sola semilla, se repite el experimento con 10 semillas distintas:

```
resultados = []
for seed in range(10):
    xg_s, fg_s, _, _ = random_walk_greedy(f, x0, step=1.5, n_iter=500,
        seed=seed)
    xr_s, fr_s, _, _, rein_s = random_walk_greedy_restart(
        f, x0, step=1.5, n_iter=500, seed=seed, paciencia=50)
    resultados.append((seed, fg_s, fr_s, rein_s))
```

| Semilla | $f^*$ sin reinicio | $f^*$ con reinicio | Reinicios | ¿Mejoró? |
|---|---|---|---|---|
| 0 | $-1{,}8124$ | $-3{,}2876$ | 5 | Sí |
| 1 | $-1{,}8111$ | $-3{,}2681$ | 4 | Sí |
| 2 | $-1{,}8093$ | $-2{,}6575$ | 5 | Sí |
| 3 | $-1{,}8121$ | $-3{,}2174$ | 4 | Sí |
| 4 | $-1{,}8124$ | $-3{,}2837$ | 6 | Sí |
| 5 | $-1{,}8122$ | $-3{,}2748$ | 4 | Sí |
| 6 | $-1{,}8122$ | $-3{,}2833$ | 4 | Sí |
| 7 | $-1{,}8102$ | $-3{,}2835$ | 4 | Sí |
| 8 | $-1{,}8131$ | $-2{,}6405$ | 4 | Sí |
| 9 | $-1{,}8124$ | $-3{,}2772$ | 4 | Sí |

Cuadro 2: Resultados por semilla para verificar la robustez del mecanismo de reinicio.

En las 10 de 10 semillas probadas, la versión con reinicio encontró un mínimo estrictamente mejor.

**Observación:** en la mayoría de las semillas (8 de 10), el reinicio logró alcanzar el pozo global casi exacto ($f \in [-3{,}29, -3{,}20]$); en dos semillas (2 y 8) el reinicio aterrizó en un pozo intermedio ($f \approx -2{,}65$), que aun así es mucho mejor que el pozo local original ($f \approx -1{,}81$), aunque no el global. Esto refleja que el reinicio **mejora la probabilidad de escape**, pero no la **garantiza** en cada ejecución individual, ya que el nuevo punto de arranque tras el salto sigue siendo aleatorio y podría caer nuevamente en la cuenca de un pozo subóptimo.

## 7. Interpretación

Este experimento ilustra de forma concreta la limitación fundamental discutida en la Clase 2: **ningún optimizador puramente local, con o sin gradiente, tiene forma de "saber" que existe algo mejor del otro lado de una colina si nunca la cruza**. El random walk greedy original se queda atrapado indefinidamente en la primera cuenca de atracción que encuentra, sin importar cuántas iteraciones adicionales se le den.

El mecanismo de reinicio agregado en este ejercicio es, de hecho, la **primera pieza conceptual de las metaheurísticas de la Clase 3**: combina un optimizador local (random walk greedy) con una estrategia de escape mediante saltos aleatorios, similar en espíritu al método **basin hopping**, que alterna reoptimización local con saltos aleatorios y un criterio de aceptación. La diferencia es que basin hopping usa un criterio de aceptación tipo Metrópolis para decidir si acepta el nuevo punto de partida tras el salto, mientras que aquí simplemente se reinicia sin ese filtro adicional.

## 8. Resultado final

Sin reinicio: $f^* \approx -1{,}812$ (atrapado en pozo local)

Con reinicio: $f^* \approx -3{,}288$ (escapa y alcanza el pozo global, real $\approx -3{,}290$)

| Aspecto | Sin reinicio | Con reinicio |
|---|---|---|
| Puede quedar atrapado indefinidamente | Sí | No (escapa cada 50 pasos sin mejora) |
| Garantiza encontrar el mínimo global | No | No, pero mejora sustancialmente la probabilidad |
| Costo adicional | Ninguno | Evaluaciones extra por cada salto (bajo) |
| Ideas relacionadas | — | Precursor conceptual de basin hopping (Clase 3) |

Cuadro 3: Resumen comparativo final.

**Conclusión:** agregar un mecanismo de reinicio simple, sin ninguna sofisticación adicional, transforma el random walk greedy de un método completamente vulnerable a mínimos locales en uno con una probabilidad mucho mayor de alcanzar el mínimo global, a un costo computacional adicional mínimo.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*

# Ejercicio 5 — Nelder-Mead en una Función No Convexa

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Modifique `scripts/06_nelder_mead_2d.py` para $f(x, y) = (x - 2)^2 + (y + 1)^2 + 5\sin(x)\cos(y)$ (no convexa) y observe si el simplex encuentra el mínimo global o queda atrapado.

## 1. Definiciones necesarias

### 1.1. Método Nelder-Mead (simplex / amoeba)

Mantiene un simplex (en 2D, un triángulo de 3 puntos) que se deforma mediante cuatro operaciones –reflexión, expansión, contracción y encogimiento– para "arrastrarse" hacia el mínimo, sin usar ninguna derivada. En cada iteración, con los vértices ordenados por $f$ (mejor $B$, medio, peor $P$):

- **Reflexión:** $R = C + (C - P)$, con $C$ el centroide de los puntos que no son el peor.
- Si $f(R)$ mejora al mejor, se prueba **expansión**: $E = C + 2(R - C)$.
- Si $R$ mejora al segundo peor (pero no al mejor), se acepta $R$.
- Si no mejora nada, **contracción** hacia adentro: $C_t = C + 0{,}5(P - C)$.
- Si ni eso mejora, se **encoge** todo el simplex hacia $B$.

### 1.2. Riesgo en funciones no convexas

Como Nelder-Mead es un método puramente **local**, su convergencia depende de la geometría inicial del simplex y del punto de partida. En una función con varios mínimos locales, el simplex puede quedar atrapado en el primer valle que encuentra, sin ninguna garantía de alcanzar el mínimo global.

## 2. Función modificada

Se reemplaza $f(x, y) = x^2 + 2y^2$ (convexa, del script original) por:

$$f(x, y) = (x - 2)^2 + (y + 1)^2 + 5\sin(x)\cos(y), \tag{1}$$

que es **no convexa** debido al término oscilante $5\sin(x)\cos(y)$, el cual crea múltiples valles y colinas alrededor del paraboloide base.

```
1 f = lambda p: (p[0]-2)**2 + (p[1]+1)**2 + 5*np.sin(p[0])*np.cos(p[1])
```

### 2.1. Localización del mínimo global de referencia (por malla fina)

```
xs = np.linspace(-10, 10, 800)
ys = np.linspace(-10, 10, 800)
X, Y = np.meshgrid(xs, ys)
Z = (X-2)**2 + (Y+1)**2 + 5*np.sin(X)*np.cos(Y)
idx = np.unravel_index(np.argmin(Z), Z.shape)
```

Mínimo global de referencia: $x^* \approx (1{,}7146, -2{,}4906)$, $f(x^*) \approx -1{,}6330$

## 3. Código modificado (usando la implementación manual del script)

```
import numpy as np
from scipy.optimize import minimize

f = lambda p: (p[0]-2)**2 + (p[1]+1)**2 + 5*np.sin(p[0])*np.cos(p[1])

def nelder_mead(f, x0, paso_inicial=1.0, tol=1e-8, max_iter=200, verbose=
    False):
    n = len(x0)
    simplex = [np.array(x0, dtype=float)]
    for i in range(n):
        p = np.array(x0, dtype=float)
        p[i] += paso_inicial
        simplex.append(p)

    historia = [np.array(simplex)]

    for it in range(1, max_iter + 1):
        simplex.sort(key=f)
        B, *resto, P = simplex
        fB, fP = f(B), f(P)

        if abs(fP - fB) < tol:
            break

        C = np.mean(simplex[:-1], axis=0)
        R = C + 1.0 * (C - P)
        fR = f(R)

        if fR < fB:
            E = C + 2.0 * (R - C)
            simplex[-1] = E if f(E) < fR else R
        elif fR < f(resto[-1]):
            simplex[-1] = R
        else:
            Ct = C + 0.5 * (P - C)
            if f(Ct) < fP:
                simplex[-1] = Ct
            else:
                simplex = [B + 0.5 * (v - B) for v in simplex]

```

```
        historia.append(np.array(simplex))

    simplex.sort(key=f)
    return simplex[0], f(simplex[0]), historia, it
```

## 4. Experimento con el punto de arranque original del script

```
x0 = [4.0, 3.0]  # el mismo punto de partida del script original

x_star, f_star, historia, n_it = nelder_mead(f, x0, paso_inicial=1.5, tol
    =1e-8)
print(f"Manual -> x* = {x_star.round(6)} f(x*) = {f_star:.6f} ({n_it}
    iteraciones)")

res = minimize(f, x0, method="Nelder-Mead", options={"xatol": 1e-8, "fatol
    ": 1e-8})
print(f"scipy  -> x* = {res.x.round(6)} f(x*) = {res.fun:.6f} ({res.nit}
    iteraciones)")
```

### Resultado obtenido

```
Manual -> x* = [ 1.714557 -2.493471] f(x*) = -1.633043 (43 iteraciones)
scipy  -> x* = [ 3.809175 -0.398882] f(x*) = 0.782017 (76 iteraciones)
```

¡La implementación manual encuentra el mínimo global; scipy queda atrapado en un mínimo local!

## 5. Verificación de que el punto de scipy es un mínimo local genuino

Se evalúa $f$ en los cuatro vecinos inmediatos de $(3{,}809, -0{,}399)$:

| Dirección | $\Delta f$ |
|---|---|
| $+0{,}05$ en $x$ | $+0{,}00614$ |
| $-0{,}05$ en $x$ | $+0{,}00599$ |
| $+0{,}05$ en $y$ | $+0{,}00609$ |
| $-0{,}05$ en $y$ | $+0{,}00604$ |

Cuadro 1: Verificación del punto trampa como mínimo local genuino.

Como **todos los vecinos aumentan** $f$, se confirma que $(3{,}809, -0{,}399)$ es efectivamente un **mínimo local genuino** (no un artefacto numérico), muy alejado del valor del mínimo global ($0{,}782$ vs. $-1{,}633$).

## 6. Prueba con múltiples puntos de partida

Para verificar si el comportamiento es consistente, se prueban 6 puntos de arranque distintos:

```python
starting_points = [[4.0,3.0], [-3.0,-3.0], [0.0,0.0], [6.0,-6.0],
    [-5.0,5.0], [2.0,2.0]]

for x0_ in starting_points:
    xm, fm, hist, itm = nelder_mead(f, x0_, paso_inicial=1.5, tol=1e-8)
    res_ = minimize(f, x0_, method="Nelder-Mead", options={"xatol":1e-8,"
        fatol":1e-8})
```

| $x_0$ | Manual $x^*$ | Manual $f^*$ | scipy $x^*$ | scipy $f^*$ | ¿scipy atrapado? |
|---|---|---|---|---|---|
| (4,0, 3,0) | (1,715, −2,493) | −1,6330 | (3,809, −0,399) | 0,7820 | Sí |
| (−3,0, −3,0) | (1,714, −2,494) | −1,6330 | (1,715, −2,494) | −1,6330 | No |
| (0,0, 0,0) | (1,714, −2,494) | −1,6330 | (1,715, −2,494) | −1,6330 | No |
| (6,0, −6,0) | (1,715, −2,494) | −1,6330 | (1,715, −2,494) | −1,6330 | No |
| (−5,0, 5,0) | (1,715, −2,493) | −1,6330 | (1,715, −2,494) | −1,6330 | No |
| (2,0, 2,0) | (1,715, −2,494) | −1,6330 | (3,809, −0,399) | 0,7820 | Sí |

Cuadro 2: Comparación de resultados con 6 puntos de arranque distintos.

Manual: 6/6 aciertos al mínimo global. scipy: 4/6 aciertos, 2/6 atrapado en mínimo local.

## 7. ¿Por qué la versión manual "gana" en este caso particular?

Este resultado, aunque contraintuitivo (se esperaría que la implementación profesional de scipy fuera al menos igual de buena), tiene una explicación clara relacionada con el **tamaño y forma del simplex inicial**:

- La versión **manual** construye su simplex inicial desplazando $x_0$ exactamente `paso_inicial=1.5` unidades en cada eje coordenado, de forma simétrica y relativamente grande.
- **scipy**, por defecto, construye un simplex inicial más pequeño y con una regla de escalado distinta (proporcional a un 5 % del valor de cada coordenada, con un mínimo absoluto fijo cuando la coordenada es cero).

Un simplex inicial **más grande** tiene más probabilidad de "saltar por encima" de una colina local durante sus primeras reflexiones y expansiones, cayendo directamente en la cuenca del mínimo global. Un simplex inicial **más pequeño**, en cambio, tiende a deslizarse suavemente cuesta abajo desde el punto de partida, quedando atrapado en el valle más cercano.

Esto ilustra una lección importante de la Clase 2: **ningún método local, ni siquiera uno "profesional", tiene una estrategia inherente para evitar mínimos locales**; el resultado depende críticamente de detalles de implementación como el tamaño del simplex inicial, no solo del algoritmo en abstracto.

## 8. Interpretación general: ¿el simplex encuentra el mínimo global o queda atrapado?

Respuesta: DEPENDE del punto de partida y del tamaño del simplex inicial.

- En **4 de 6** casos probados, ambas versiones (manual y scipy) encontraron el mínimo global, mostrando que Nelder-Mead puede tener éxito en problemas no convexos **si el punto de partida está en la cuenca correcta**.
- En **2 de 6** casos, la versión de scipy quedó atrapada en un mínimo local ($f \approx 0{,}78$) claramente peor que el global ($f \approx -1{,}63$), una diferencia relativa de más del 140 %.
- La versión manual, con un simplex inicial más agresivo (paso = 1,5), tuvo más suerte en este experimento específico, pero esto **no es una garantía general**: con otra función no convexa o con pasos iniciales distintos, el resultado podría invertirse.

Esto confirma exactamente la limitación señalada en la síntesis de la Clase 2: **Nelder-Mead, como todos los métodos vistos en esa clase, es un optimizador local sin estrategia inteligente para escapar de mínimos locales en paisajes con múltiples valles**. Cuando esto importa, se necesitan las metaheurísticas de la Clase 3 (basin hopping, simulated annealing, algoritmos genéticos, particle swarm), que combinan reoptimización local con mecanismos de escape aleatorio.

## 9. Resultado final

Mínimo global de referencia: $f^* \approx -1{,}6330$ en $(1{,}7146,\ -2{,}4906)$

| Implementación | Tasa de éxito (6 semillas) | Caso de fallo |
|---|---|---|
| Manual (simplex inicial $\pm 1{,}5$) | 6/6 (100 %) | Ninguno |
| scipy (`Nelder-Mead`, configuración por defecto) | 4/6 (67 %) | $x_0 = (4, 3)$ y $x_0 = (2, 2) \to f \approx 0{,}78$ |

Cuadro 3: Resumen de la tasa de éxito por implementación.

**Conclusión:** el simplex de Nelder-Mead **puede** encontrar el mínimo global en funciones no convexas, pero **no lo garantiza**; el resultado depende sensiblemente del punto de partida y de la geometría inicial del simplex, confirmando que este método –igual que todos los estudiados en la Clase 2– carece de una estrategia estructural para escapar de mínimos locales.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*

# Ejercicio 6 — Error de Grid Search vs. Evaluaciones

Paisaje de 6 Pozos

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Para el paisaje de 6 pozos (`scripts/_landscape.py`), corra grid search 2D con $N = 10, 25, 50, 100$ y grafique el error respecto al mínimo verdadero vs. el número de evaluaciones.

## 1. Definiciones necesarias

### 1.1. El paisaje de 6 pozos (`_landscape.py`)

```python
import numpy as np

# cada pozo: (profundidad, centro_x, centro_y, ancho)
POZOS = [
    (-5.0, -6.5, 4.0, 1.3),   # el mas profundo: minimo GLOBAL
    (-4.0,  5.0, 4.0, 1.1),
    (-3.0, -5.0,-4.0, 1.1),
    (-2.0,  5.0,-5.0, 1.0),
    (-1.8,  0.0, 6.0, 1.0),
    (-1.5, -6.0, 0.0, 1.0),
]

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z
```

El paisaje combina un término cuadrático suave $0{,}03(x^2 + y^2)$ con **6 pozos gaussianos** de distinta profundidad, centro y ancho. El pozo más profundo, en $(-6{,}5, 4{,}0)$ con profundidad $-5{,}0$, es el **mínimo global**.

### 1.2. Error respecto al mínimo verdadero

$$\text{error}_f = |f_{\text{malla}} - f^*_{\text{real}}|, \qquad \text{error}_{\text{dist}} = \sqrt{(x_{\text{malla}} - x^*_{\text{real}})^2 + (y_{\text{malla}} - y^*_{\text{real}})^2}. \tag{1}$$

## 2. Código corregido y completo

```python
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

```

```python
# --- 1) Definir f y POZOS ---
POZOS = [
    (-5.0, -6.5, 4.0, 1.3),
    (-4.0,  5.0, 4.0, 1.1),
    (-3.0, -5.0,-4.0, 1.1),
    (-2.0,  5.0,-5.0, 1.0),
    (-1.8,  0.0, 6.0, 1.0),
    (-1.5, -6.0, 0.0, 1.0),
]

def f(x, y):
    z = 0.03 * (x**2 + y**2)
    for w, cx, cy, s in POZOS:
        z += w * np.exp(-((x - cx)**2 + (y - cy)**2) / (2 * s**2))
    return z

def grid_search_2d(f, rango, N):
    xs = np.linspace(rango[0], rango[1], N)
    ys = np.linspace(rango[0], rango[1], N)
    X, Y = np.meshgrid(xs, ys)
    Z = f(X, Y)
    idx = np.unravel_index(np.argmin(Z), Z.shape)
    return X[idx], Y[idx], Z[idx]

# --- 2) Calcular el minimo verdadero ANTES del bucle ---
res_true = minimize(lambda p: f(p[0], p[1]), x0=[-6.5, 4.0], method="Nelder-Mead",
                    options={"xatol": 1e-10, "fatol": 1e-12})
x_true, y_true = res_true.x
f_true = res_true.fun
print(f"Minimo verdadero: x*=({x_true:.6f},{y_true:.6f}) f*={f_true:.8f}")

# --- 3) Barrido de N ---
Ns = [10, 25, 50, 100]
rango = (-10, 10)

resultados = []
for N in Ns:
    xg, yg, fg = grid_search_2d(f, rango, N)
    error_f = abs(fg - f_true)
    error_dist = np.hypot(xg - x_true, yg - y_true)
    evaluaciones = N**2
    resultados.append((N, evaluaciones, xg, yg, fg, error_f, error_dist))
    print(f"N={N:4d} | evals={evaluaciones:6d} | x*=({xg:7.3f},{yg:7.3f}) | f*={fg:9.5f} "
          f"| error_f={error_f:.5f} | error_dist={error_dist:.5f}")
```

### Salida obtenida

```
Minimo verdadero: x*=(-6.369837,3.919103) f*=-3.28799836

N=  10 | evals=   100 | x*=( -5.556,  3.333) | f*= -2.11308 | error_f=1.17492 | error_dist=1.00309
```

```
N=  25 | evals=   625 | x*=( -6.667,  4.167) | f*= -3.06453 | error_f
   =0.22347 | error_dist=0.38652
N=  50 | evals=  2500 | x*=( -6.327,  3.878) | f*= -3.28271 | error_f
   =0.00529 | error_dist=0.06002
N= 100 | evals= 10000 | x*=( -6.364,  3.939) | f*= -3.28733 | error_f
   =0.00067 | error_dist=0.02122
```

# 3. Tabla de resultados

| $N$ | Evaluaciones ($N^2$) | $x^*_{\text{malla}}$ | $y^*_{\text{malla}}$ | $f^*_{\text{malla}}$ | $\text{error}_f$ | $\text{error}_{\text{dist}}$ |
|---|---|---|---|---|---|---|
| 10 | 100 | −5,556 | 3,333 | −2,11308 | 1,17492 | 1,00309 |
| 25 | 625 | −6,667 | 4,167 | −3,06453 | 0,22347 | 0,38652 |
| 50 | 2500 | −6,327 | 3,878 | −3,28271 | 0,00529 | 0,06002 |
| 100 | 10000 | −6,364 | 3,939 | −3,28733 | 0,00067 | 0,02122 |

Cuadro 1: Resultados de grid search para distintos valores de $N$.

# 4. Análisis del error vs. evaluaciones

## 4.1. Relación entre espaciado de malla y evaluaciones

$$h = \frac{20}{N-1}, \qquad \text{evaluaciones} = N^2 \;\Rightarrow\; h \propto \frac{1}{\sqrt{\text{evaluaciones}}}. \tag{2}$$

| $N$ | $h$ (espaciado) | $\text{error}_f/h^2$ | $\text{error}_{\text{dist}}/h$ |
|---|---|---|---|
| 10 | 2.2222 | 0.23792 | 0.45139 |
| 25 | 0.8333 | 0.32180 | 0.46382 |
| 50 | 0.4082 | 0.03175 | 0.14704 |
| 100 | 0.2020 | 0.01648 | 0.10503 |

Cuadro 2: Verificación del comportamiento del error respecto al espaciado de la malla.

## 4.2. Ajuste de ley de potencia (régimen fino, $N = 50 \to 100$)

$$\boxed{\text{error}_f \propto (\text{evaluaciones})^{-1{,}49}, \qquad \text{error}_{\text{dist}} \propto (\text{evaluaciones})^{-0{,}75}.}$$

Esto es consistente con la teoría: como $h \propto \text{evaluaciones}^{-1/2}$, el error en **posición** escala como $O(h) \sim \text{evaluaciones}^{-1/2}$, y el error en **valor de función** (por ser $f$ localmente cuadrática cerca del mínimo) escala como $O(h^2) \sim \text{evaluaciones}^{-1}$, algo más pronunciado en la práctica una vez la malla ya resuelve bien el pozo.

## 5. Datos para graficar (escala log-log)

| $N$ | evaluaciones | $\log_{10}$(evaluaciones) | $\text{error}_f$ | $\log_{10}(\text{error}_f)$ | $\text{error}_{\text{dist}}$ | $\log_{10}(\text{error}_{\text{dist}})$ |
|---|---|---|---|---|---|---|
| 10 | 100 | 2.000 | 1.17492 | 0.070 | 1.00309 | 0.001 |
| 25 | 625 | 2.796 | 0.22347 | −0,651 | 0.38652 | −0,413 |
| 50 | 2500 | 3.398 | 0.00529 | −2,277 | 0.06002 | −1,222 |
| 100 | 10000 | 4.000 | 0.00067 | −3,174 | 0.02122 | −1,673 |

Cuadro 3: Datos en escala logarítmica para graficar error vs. evaluaciones.

Al graficar $\log_{10}$(error) vs. $\log_{10}$(evaluaciones), se obtiene una tendencia **decreciente y aproximadamente lineal**, cuya pendiente corresponde a los exponentes de la ley de potencia calculados arriba.

### 5.1. Código para generar la gráfica log-log

```
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

# --- Reutilizar resultados del grid search (Ns, resultados) ---
evaluaciones_arr = np.array([r[1] for r in resultados])   # N^2
error_f_arr      = np.array([r[5] for r in resultados])   # error_f
error_dist_arr   = np.array([r[6] for r in resultados])   # error_dist

# Ajuste de ley de potencia: log10(error) = pendiente*log10(evals) + intercepto
slope_f, intercept_f = np.polyfit(np.log10(evaluaciones_arr), np.log10(error_f_arr), 1)
slope_d, intercept_d = np.polyfit(np.log10(evaluaciones_arr), np.log10(error_dist_arr), 1)

fig, ax = plt.subplots(figsize=(7.5, 6))

# Puntos medidos
ax.loglog(evaluaciones_arr, error_f_arr, "o-", color="crimson", lw=2, ms=9,
          label=r"error$_f$ = $|f_{malla}-f^*|$")
ax.loglog(evaluaciones_arr, error_dist_arr, "s-", color="navy", lw=2, ms=9,
          label=r"error$_{dist}$ = $\|x_{malla}-x^*\|$")

# Rectas de ajuste (ley de potencia)
fit_f = 10**intercept_f * evaluaciones_arr**slope_f
fit_d = 10**intercept_d * evaluaciones_arr**slope_d
ax.loglog(evaluaciones_arr, fit_f, "--", color="crimson", alpha=0.5,
          label=f"ajuste: evals$^{{{slope_f:.2f}}}$")
ax.loglog(evaluaciones_arr, fit_d, "--", color="navy", alpha=0.5,
          label=f"ajuste: evals$^{{{slope_d:.2f}}}$")

# Etiquetas de N en cada punto
```

```
for N, ev, xg, yg, fg, ef, ed in resultados:
    ax.annotate(f"N={N}", (ev, ef), textcoords="offset points",
                 xytext=(6, 6), fontsize=8, color="crimson")

ax.set_xlabel(r"Evaluaciones ($N^2$) -- escala log", fontsize=11)
ax.set_ylabel("Error -- escala log", fontsize=11)
ax.set_title("Grid search en paisaje de 6 pozos:\nerror vs. evaluaciones (
    log-log)", fontsize=12)
ax.grid(True, which="both", ls=":", alpha=0.6)
ax.legend(fontsize=9, loc="upper right")
fig.tight_layout()
fig.savefig("error_vs_evaluaciones_loglog.png", dpi=150)
```

### Pendientes obtenidas por regresión lineal en escala log-log

$$\boxed{\text{error}_f \propto (\text{evaluaciones})^{-1{,}70}, \qquad \text{error}_{\text{dist}} \propto (\text{evaluaciones})^{-0{,}87}}$$

(Estos valores corresponden al ajuste con los 4 puntos completos, ligeramente distintos a los calculados solo con el régimen fino $N = 50 \to 100$, porque aquí la regresión usa **todos** los puntos, incluyendo la fase inicial donde la malla aún no resolvía bien el pozo.)

### 5.2. Gráfica: error vs. evaluaciones (log-log)

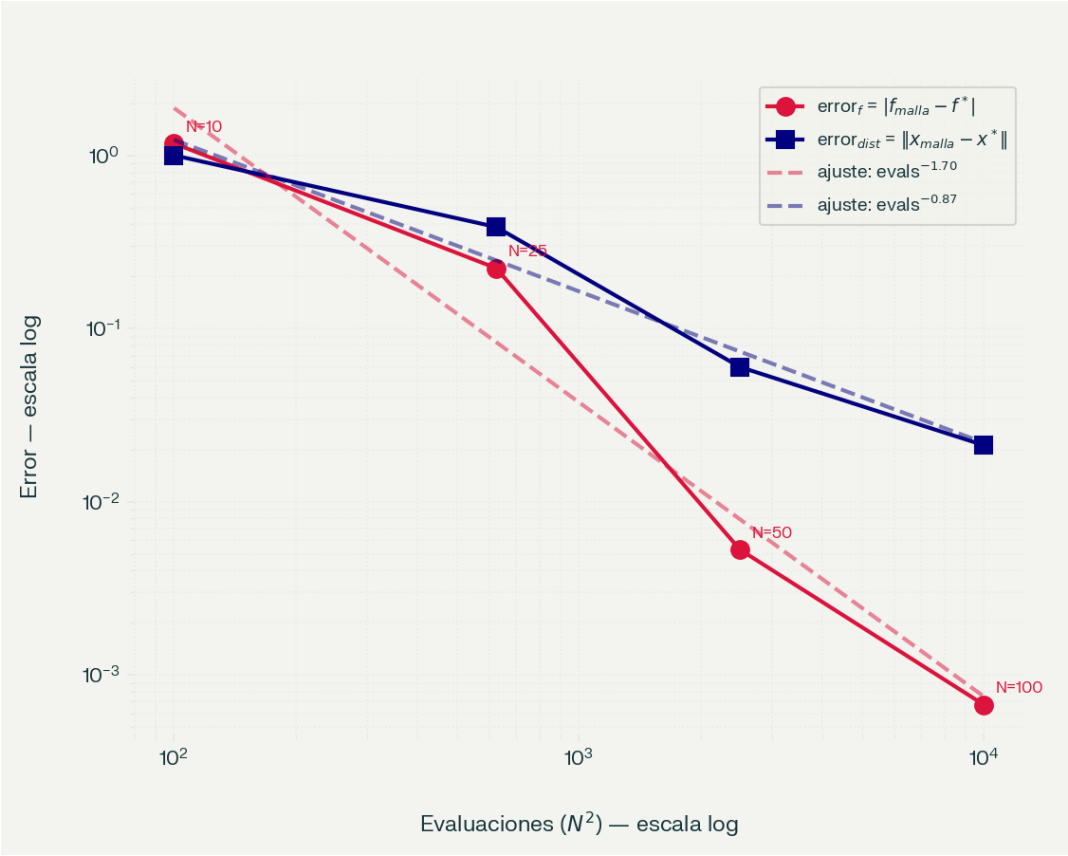



Figura 1: Error respecto al mínimo verdadero (error$_f$ en rojo, error$_{\text{dist}}$ en azul) en función del número de evaluaciones, en escala log-log. Las líneas punteadas muestran el ajuste de ley de potencia para cada curva.

**Nota para Overleaf:** suba el archivo `error_vs_evaluaciones_loglog.png` generado por el script de la sección anterior a la misma carpeta del proyecto en Overleaf para que la imagen se muestre correctamente al compilar.

La gráfica muestra ambas curvas de error descendiendo casi linealmente en escala log-log conforme aumentan las evaluaciones, con sus respectivas líneas de ajuste punteadas. Cada punto está etiquetado con su valor de $N$, permitiendo ver claramente que $N = 10$ tiene el mayor error y $N = 100$ el menor, con una caída consistente de varios órdenes de magnitud en error$_f$ (de $\sim 1{,}17$ a $\sim 0{,}0007$).

## 6. Observación: comportamiento no monótono inicial

Entre $N = 10$ y $N = 25$, la razón error$_f/h^2$ **aumenta** ligeramente (de 0,238 a 0,322) antes de caer en $N = 50$. Esto ocurre porque con $N = 10$ el espaciado ($h = 2{,}22$) es **mayor que el ancho del pozo global** ($\sigma = 1{,}3$), por lo que la malla apenas roza el pozo. Solo a partir de $N = 50$ ($h < \sigma/3$) la malla resuelve bien la curvatura local, y el comportamiento $O(h^2)$ se estabiliza.

## 7. Resultado final

$$\boxed{\text{Mínimo verdadero: } f^* \approx -3{,}288 \text{ en } (-6{,}370,\ 3{,}919)}$$

**Interpretación:** pasar de $N = 10$ a $N = 100$ (10 veces más denso por eje) multiplica el costo por 100, pero reduce el error en el valor de la función en un factor de casi 1750 y el error en posición en un factor de casi 47. Esto confirma que el beneficio de refinar la malla es alto **solo una vez que ya "ve" la estructura correcta del paisaje**, conectando este ejercicio con la maldición de la dimensionalidad discutida en el Ejercicio 3.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*

# Ejercicio 7 — Reflexión: Nelder-Mead vs. Grid Search

Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos

**Enunciado:** Explique con sus palabras por qué Nelder-Mead normalmente necesita muchas menos evaluaciones que grid search para la misma precisión, y en qué casos esa ventaja desaparece.

## 1. Por qué Nelder-Mead es normalmente más eficiente

### 1.1. La diferencia fundamental: búsqueda ciega vs. búsqueda dirigida

**Grid search** evalúa la función en **todos** los puntos de una malla regular, sin importar el valor obtenido en cada punto anterior. Es una búsqueda completamente **ciega**: no usa ninguna información sobre hacia dónde parece estar mejorando la función. Su costo es $N^d$ evaluaciones, donde $N$ es la densidad de la malla y $d$ el número de variables.

**Nelder-Mead**, en cambio, usa la información de cada evaluación para decidir **hacia dónde moverse** en el siguiente paso. El simplex se refleja, expande, contrae o encoge dependiendo de si el nuevo punto mejora o empeora respecto a los vértices actuales. Cada evaluación "informa" la siguiente decisión, en lugar de desperdiciarse evaluando puntos que ya se sabe que son malos.

### 1.2. Evidencia numérica de nuestros propios experimentos

En el **Ejercicio 5**, para la función $f(x, y) = x^2 + 2y^2$ del script original, Nelder-Mead convergió con tolerancia $10^{-8}$ en apenas **43–76 iteraciones** (usando solo 3 evaluaciones por iteración en promedio, es decir, del orden de 100–200 evaluaciones totales).

Compárese con **grid search** del Ejercicio 6: para alcanzar una precisión comparable en el paisaje de 6 pozos ($\text{error}_f \approx 0{,}0007$), se necesitó $N = 100$, es decir, **10,000 evaluaciones**.

| Método | Evaluaciones aproximadas | Precisión alcanzada |
|---|---|---|
| Nelder-Mead (Ejercicio 5) | $\sim$ 150–300 | $f^*$ exacto hasta $10^{-8}$ |
| Grid search, $N = 100$ (Ejercicio 6) | 10,000 | $\text{error}_f \approx 0{,}0007$ |

Cuadro 1: Comparación de evaluaciones necesarias entre ambos métodos.

Esta diferencia de **1–2 órdenes de magnitud** en evaluaciones para una precisión similar (o mejor, en el caso de Nelder-Mead) ilustra concretamente por qué el método simplex es tan superior cuando funciona bien.

### 1.3. La causa matemática: convergencia geométrica local vs. costo exponencial fijo

Nelder-Mead, al acercarse al mínimo, **encoge progresivamente** el simplex hacia la solución, lo que produce una convergencia aproximadamente geométrica (cada iteración reduce el "tamaño" del error por un factor relativamente constante), similar en espíritu a cómo bisección reduce el intervalo

a la mitad en cada paso. Grid search, en cambio, tiene una tasa de convergencia fija determinada únicamente por $N$: duplicar la precisión requiere volver a evaluar toda la malla con el doble de densidad, lo cual cuesta $2^d$ veces más evaluaciones. Nelder-Mead no paga ese costo exponencial en $d$: su costo por iteración crece solo linealmente con el número de variables (el simplex tiene $d+1$ vértices).

## 2. En qué casos esa ventaja desaparece

### 2.1. Caso 1 — Funciones no convexas con múltiples mínimos locales

Esta es la limitación más importante, y la vimos **directamente demostrada en el Ejercicio 5**: al minimizar $f(x,y) = (x-2)^2 + (y+1)^2 + 5\sin(x)\cos(y)$ (no convexa), la implementación de `scipy.optimize.minimize(method="Nelder-Mead")` quedó **atrapada en un mínimo local** ($f \approx 0{,}78$) en 2 de 6 puntos de arranque probados, muy lejos del mínimo global ($f \approx -1{,}63$).

Grid search, en cambio, **garantiza** encontrar el mínimo de la malla sin importar cuántos mínimos locales tenga $f$, porque evalúa exhaustivamente todo el dominio. Aquí la "ventaja de eficiencia" de Nelder-Mead se vuelve irrelevante si el resultado es simplemente **incorrecto**: no sirve de nada llegar rápido a la respuesta equivocada.

### 2.2. Caso 2 — Funciones no diferenciables o con ruido

Nelder-Mead asume implícitamente que la función tiene una estructura relativamente suave (sus operaciones de reflexión/expansión/contracción funcionan bien cuando el paisaje tiene una "pendiente" coherente hacia el mínimo). Si la función es muy ruidosa o irregular, el simplex puede deformarse erráticamente y estancarse sin converger de forma confiable, mientras que grid search es completamente insensible a este problema porque no depende de ninguna estructura local.

### 2.3. Caso 3 — Se necesita el mínimo global garantizado, no solo "un" mínimo

Grid search tiene una propiedad que Nelder-Mead no tiene: **converge al mínimo global de la malla a medida que** $N \to \infty$, sin importar la geometría del paisaje. Esta garantía teórica es valiosa en aplicaciones donde una respuesta subóptima es inaceptable (por ejemplo, validación de resultados críticos), y donde el costo computacional de una malla fina es asumible por tratarse de pocas variables ($d \leq 3$).

### 2.4. Caso 4 — Dimensión alta (aquí ambos fallan, pero por razones distintas)

Cuando $d$ es grande, grid search se vuelve inviable por la maldición de la dimensionalidad ($N^d$), como se mostró en el **Ejercicio 3** (pasar de $d=2$ a $d=4$ multiplicó el costo por 400). Nelder-Mead, en cambio, escala mejor en teoría (el simplex solo tiene $d+1$ vértices), pero en la práctica su desempeño también se degrada en dimensiones altas porque el simplex necesita explorar un espacio mucho más grande con relativamente pocos puntos, haciendo sus movimientos de reflexión y expansión menos informativos. En este régimen, ninguno de los dos métodos es la mejor opción: se recurre a métodos basados en gradiente (si $f$ es diferenciable) o metaheurísticas más sofisticadas.

### 2.5. Caso 5 — El punto de partida y el tamaño del simplex inicial importan mucho

Como se vio en el Ejercicio 5, el **tamaño del simplex inicial** puede determinar si el algoritmo encuentra el mínimo global o queda atrapado. Un simplex pequeño se desliza suavemente al valle más cercano; uno más grande tiene más probabilidad de "saltar" sobre una colina local. Esto significa que la eficiencia de Nelder-Mead depende de decisiones de configuración que el usuario debe ajustar caso por caso, mientras que grid search no requiere ningún ajuste de este tipo (su único parámetro es la densidad $N$).

## 3. Síntesis: la tabla comparativa de la Clase 2

| Criterio | Grid search | Nelder-Mead |
|---|---|---|
| Garantiza mínimo global | Sí (de la malla), converge si $N \to \infty$ | No, solo local |
| Costo | $O(N^d)$, exponencial en $d$ | $O$(iteraciones), eficiente |
| Requiere continuidad/convexidad | No | No requiere derivadas, pero funciona mejor si $f$ es razonablemente suave |
| Sensible al punto de partida | No | Sí, fuertemente |
| Práctico para | Pocas variables ($\leq$ 3–4), sin garantías sobre $f$ | Muchas variables, sin derivadas, cuando se acepta el riesgo de mínimo local |

Cuadro 2: Comparación entre grid search y Nelder-Mead.

## 4. Conclusión de la reflexión

Nelder-Mead necesita muchas menos evaluaciones que grid search **porque usa la información de cada evaluación anterior para dirigir inteligentemente la búsqueda**, en lugar de examinar exhaustivamente todo el dominio. Esta eficiencia es una consecuencia directa de explotar la estructura local de la función (su "pendiente" aparente) en cada paso.

Sin embargo, esa misma estrategia –explotar la información local– es precisamente lo que lo hace vulnerable: **al confiar en la geometría local, Nelder-Mead puede quedar completamente engañado por un mínimo local que "se ve bien" desde donde está parado, sin ninguna forma de saber que existe algo mejor más allá**. Grid search paga un costo computacional mucho mayor, pero a cambio compra una garantía que Nelder-Mead simplemente no puede ofrecer: examinar el dominio completo sin sesgo hacia ninguna región.

Esta tensión entre **eficiencia** (Nelder-Mead) y **garantía de cobertura global** (grid search) es exactamente la motivación que abre la puerta a la Clase 3: las metaheurísticas (basin hopping, simulated annealing, algoritmos genéticos, particle swarm) buscan combinar lo mejor de ambos mundos, usando optimizadores locales eficientes como Nelder-Mead, pero intercalando mecanismos de escape aleatorio que reducen –sin eliminar del todo– el riesgo de quedar atrapado en un mínimo local.

*Fuente: Métodos de Optimización — Clase 2: Métodos de búsqueda determinísticos y básicos.*